Source: docs\PRODUCTS.md | Export: 11 November 2026

Product CRUD milestone

Current workflow and confirmed missing Production index: PRODUCT_WORKFLOW.md (20 October 2026). It supersedes the older form labels, mandatory optional saree details and dual-field inventory shape below.

Production catalog startup fix and release requirement: CATALOG_API_DIAGNOSTICS.md. The current Firebase Admin dependency chain requires Vercel's documented `NODE_OPTIONS=--experimental-require-module` setting and a new deployment.

Target: `parinaya-silks`, Firestore `(default)`. Local preparation only; no live products, uploads, deletions, rule publication or deployment.

Schema and trust boundary

`products/{autoId}` is canonical. Stable unique `slug` is reserved in `productSlugs/{slug}` and never changes on rename. Product writes use the authenticated `/api/catalog` server endpoint; direct browser writes remain denied. Money is integer paise (`pricePaise`, optional `originalPricePaise`); discount and stock totals are derived, not independently editable.

The model includes name, SKU, description, permanent `categoryId`, ordered verified image metadata, `coverMediaId`, simple or colour-variant inventory, low-stock threshold, optional saree details, featured/new-arrival flags and Draft/Published/Archived status. Server metadata includes `schemaVersion`, creation/update timestamps, revision and last save operation ID. Drafts may be incomplete. Publication requires name, SKU, description, enabled category, positive price and a verified cover image with photo descriptions. The editor generates a default product code and permanent link. Fabric, weave, length and care are optional.

Inventory has one source: `{mode:'simple', quantity}` or `{mode:'variants', variants}` with up to 20 unique IDs/codes, colour names and integer quantities. The inactive field is omitted. Legacy empty variants/zero simple totals remain readable but are normalized on save; actual contradictions fail. No sizes are required. Optimistic revisions and transactions reject stale edits rather than overwrite concurrent stock changes. Reservation/checkout state is not implemented; future trusted reservations must integrate this revision/inventory policy.

`productMedia/{mediaId}` records upload intent, product ownership, fixed Cloudinary public ID, verification and durable metadata. Media is restricted to one product. Product save resolves references from these trusted records. Remove/replace detaches an image; it never destroys an asset. Abandoned uploads and detached media are retained for owner review. Automatic cleanup and permanent product deletion remain unavailable until cross-record/history reference checks exist. Category deletion stays denied. Archive preserves identity and media.

Public catalog reads go through the same single API function, which returns only Published products whose category is currently enabled. This policy applies to listing and direct slug lookup, including category changes without rewriting products. Firestore direct gets enforce the same status/category policy, while public collection listing is denied (the server supplies bounded, filtered pages). Every privileged action revalidates the Firebase token and enabled admin record; saves also check the access record in the write transaction.

Final stored fields

Implementation files: `src/products/{model,client,adapter,hooks,PublicProduct}.ts(x)`; `src/admin/{ProductManager,ProductImages,AdminProductForm,AdminProducts,AdminPanel,AdminAccess}.tsx`; `src/App.tsx`, `src/types/index.ts`, `src/views/{HomeView,ShopView,ProductDetailView}.tsx`; `api/catalog.ts`; `server/{firebase,cloudinary,catalog,handler}.ts`; `scripts/catalog-dev.ts`; `.env.example`, `vite.config.ts`, `vercel.json`, `firebase.json`, `firestore.rules`, `firestore.indexes.json`, `package.json` and lockfile. Tests are in `src/products/model.test.ts`, `scripts/{cloudinary.test,catalog-emulator.test,catalog-emulator-server}.ts`, `scripts/product-browser.test.py`, with existing admin/browser runners updated. README and milestone guides point to this document.

Group	Fields
Identity	Firestore auto ID; `name` (up to 160), `slug` (up to 100, lowercase ASCII/hyphens, required even on a draft), `sku` (up to 64 ASCII letters/digits/dot/underscore/hyphen), `description` (up to 6000).
Category / price	`categoryId` (permanent category document ID, or empty on an incomplete draft), `pricePaise` (integer 0-100000000, positive for publication), `originalPricePaise` (null or an integer greater than selling price). No stored discount percentage.
Images	`images` ordered array, up to 8; each has `mediaId`, `alt` (up to 200), `assetId`, `publicId`, `url`, `width`, `height`, `bytes`, `format`, `version`. `coverMediaId` points to an image in that array. All durable metadata comes from the trusted upload registry.

Group	Fields
Inventory	`inventory: {mode, quantity, variants}`; mode is `simple` or `variants`. Variants contain `{id, sku, name, hex, quantity}`. Quantities are integers 0-1000000; up to 20 variants. `lowStockThreshold` is a nonnegative integer. Totals and in-stock state are derived.
Saree	`fabric`, `weaveType`, `zariType`, `colorName`, `colorHex`, `sareeLength`, `blouseIncluded` boolean, `blouseDetails`, `careInstructions` text, `weightGrams` null or nonnegative integer. No size fields or invented certification.
Publication	`status` = Draft, Published or Archived; `featured`, `newArrival` booleans.
Protected	`schemaVersion: 1`, `revision` (incremented transactionally), `createdAt`, `updatedAt` (server timestamps), `lastOperationId`, `lastOperationHash`. Clients cannot supply these as product fields.

`productSlugs/{slug}` stores `productId` and server `createdAt`. Reservations remain occupied after unpublishing/archive. All direct product/slug/media writes are denied in rules, including browser administrators. The Admin SDK bypasses rules, so token/access verification, category checks, media ownership and shared field validation occur in the server operation. Update transactions also read the access record and current category so revocation/category changes cannot race past an unchecked write.

Server lists use 50 records per page ordered by creation time/document ID, with a stable document cursor. Public pages skip disabled categories on each request; pagination can return an empty page with another cursor if that scanned page contained hidden records. Search, filters and metrics explicitly apply to loaded records; Load more expands that scope. Direct product lookup uses its slug reservation and does not depend on having loaded its page. Public views refresh on focus and every 30 seconds; a response never grants access based on browser storage. Already downloaded images are public Cloudinary assets and are not revoked by hiding their product.

Media workflow and retention

The browser requests a signature with only product ID, media ID, MIME type and byte size. The server chooses the public ID `parinaya-silks/products/{productId}/{mediaId}`, timestamp, signed preset, JPEG/PNG/WebP format allowlist and `overwrite=false`. Arbitrary upload parameters are rejected. An authenticated admin may request up to 100 new upload intents per UTC day. A fixed-ID retry within ten minutes reuses its intent; it cannot overwrite an existing asset. Cloudinary upload signatures themselves follow Cloudinary's validity window; the intent retry window does not revoke a signature already issued.

Uploads go directly to Cloudinary with progress. Retry first attempts metadata verification so a lost response does not cause a duplicate upload. After upload, the server uses authenticated Cloudinary Admin API lookup of the fixed image/upload public ID. It checks resource/delivery type, identifier, format, dimensions (1-20000), size (up to 10 MiB), non-animated asset, version, asset ID and HTTPS delivery host before marking the registry verified. Save resolves image references against that registry; a browser-supplied URL or another product's media is not accepted. Metadata lookup failures are recoverable without deleting or re-uploading an already present asset.

Reorder, cover, alt text and removal are saved with the product. To replace, upload the new asset and remove the old entry (remove first if already at eight). No image transformations, generated substitutes, stretching or destructive media API is used. Images render with `object-contain`. Pending/failed uploads are excluded from saved images; saving waits while an upload is active. A failed save keeps the draft and verified images for retry. Dismissing a failed upload or leaving the form retains the intent and any asset. A trusted owner can review unverified/unused registry records against Cloudinary later; there is intentionally no automatic cleanup that might delete referenced media.

Required environment and local commands

Keep the existing `VITE_FIREBASE_*` browser configuration, including project `parinaya-silks` and database `(default)`. No server secret may use a `VITE_` prefix.

Create an **ignored** `.env.server.local` for `npm run dev:api` with:

```
# Local approved credential file outside the checkout, OR use server JSON below.
GOOGLE_APPLICATION_CREDENTIALS=C:/path/outside-repo/approved-service-account.json
# Alternative for Vercel: the complete service-account JSON as a protected value.
# FIREBASE_SERVICE_ACCOUNT_JSON={...}
CLOUDINARY_CLOUD_NAME=your_parinaya_cloud
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_private_api_secret
CLOUDINARY_UPLOAD_PRESET=your_signed_product_preset
```

Use the already approved PARINAYA Firebase credential, not another client's account. The server explicitly targets `parinaya-silks` and rejects conflicting project variables or credential project IDs. The identity needs Firebase Auth user-read/token-revocation checking and Firestore read/write permissions. ADC is supported on a trusted workstation when neither credential variable is set. Do not paste keys into chat, commit credential files, or put the private JSON/API secret in frontend environment variables. `.env.example` contains placeholders only; both actual local environment files are ignored.

In the PARINAYA Cloudinary account create a **signed** product upload preset restricting images to JPEG, PNG and WebP, maximum 10 MiB, with no incoming transformations, overwriting, automatic filename/public-ID prefix changes or generated imagery. The fixed public ID must be honored. Do not enable an unsigned preset for this workflow. Media type, maximum size and identifier checks also run after upload before persistence. No Cloudinary account/configuration was available during implementation, and no live upload was attempted.

Run in two terminals:

```
npm.cmd run dev:api
npm.cmd run dev
```

The local API binds `127.0.0.1:3200`; Vite on 3000 proxies `/api` there. Starting Vite alone does NOT start the API. A static production preview also needs an API proxy/server; it does not execute Vercel functions by itself. Server configuration changes require restarting the API.

On Vercel set the same four `CLOUDINARY_*` variables and these three individual server credentials from the approved service-account JSON: `FIREBASE_PROJECT_ID` = its `project_id` (`parinaya-silks`), `FIREBASE_CLIENT_EMAIL` = its `client_email`, and `FIREBASE_PRIVATE_KEY` = its `private_key`. The client email is the service-account email, not the administrator's Gmail login. Paste field values without surrounding JSON quotes; literal `\n` sequences in the private key are normalized by the server. These three credentials take precedence when supplied; incomplete credentials fail closed. Keep all three server-only and never prefix them with `VITE_`.

Protected `FIREBASE_SERVICE_ACCOUNT_JSON` containing the complete approved credential remains an alternative, so existing JSON-based installations continue working. A workstation credential path is not usable on Vercel. Keep the existing browser Firebase variables. The single new `api/catalog.ts` Node function handles public GET listing/detail and authenticated POST actions (`list`, `get`, `save`, `sign`, `verify`; `delete` is explicitly rejected). It returns private/no-store responses and sanitized errors; uploaded bytes never pass through the function. `maxDuration` is 30 seconds. Firebase Admin SDK is now a runtime dependency, pinned to the existing version. Release the updated server code and redeploy after adding or renaming environment variables.

No `.vercel` project link, Vercel CLI or authenticated plan information was available locally, so the actual account plan remains unverified. The implementation adds only **one** function and avoids background work or large upload bodies. Owner should confirm the existing project's function settings before release. See Vercel function limits and Node runtimes. Cloudinary references: signed uploads, signature rules, Admin API.

Rules, index and release

Review `firestore.rules` and `firestore.indexes.json` against current live integrations before publishing. They preserve category writes/uniqueness, dual administrator permission, access-record/private collection restrictions and category deletion denial. Product public direct gets require Published plus a currently enabled category; public collection queries are denied, since public listing is mediated by the API. Product/media/slug browser writes remain denied. Other business modules stay unconnected.

The new composite index is `products: status ASC, createdAt DESC` (document-ID descending is the implicit tie-breaker). Admin ordering uses the normal single-field creation-time index. Owner publication commands, NOT run by this task:

```
npx.cmd firebase deploy --only firestore:rules,firestore:indexes --project parinaya-silks
```

Wait for the index to become ready. Reconcile any existing live rules/index definitions instead of overwriting them blindly; the supplied historical permissive test grant must not remain. Review existing products/media before release: schema-v1 records and matching slug reservations are expected, and no legacy records are migrated automatically. Unknown schemas fail with an owner-review message, not a sample fallback.

Set server variables, then perform the owner's normal commit/push/Vercel release. API paths are already excluded from the SPA rewrite. No live settings, rules, data, commits, pushes or deployments were changed here.

Tests and acceptance

```
npm.cmd run lint
npm.cmd test
npm.cmd run build
npm.cmd run test:admin
```

The emulator suite tests real product transactions, real Auth emulator tokens, browser-write denial, publication/category privacy, unique slugs, stale stock races, idempotent retries, variants, verified-media ownership/retention, rejected deletion and existing admin/category regressions. Browser product checks use the real loopback API/SDK and **mocked Cloudinary upload/metadata responses**. They do not prove live Cloudinary account settings, uploads, API permissions or production index readiness. Test-only Cloudinary fixtures are confined to scripts and cannot be enabled in the production API. Screenshots go to ignored `.cache/products`.

Validation completed on 17 October 2026: TypeScript check, all 37 unit/Cloudinary-adapter tests, production build, full admin/category/product emulator suite and rendered product browser workflow passed. Existing storefront regression checks passed against local development and production preview; intercepted Firebase checks and all 192 route/viewport checks passed. Mobile/desktop product form and detail screenshots were visually reviewed using

synthetic images. The build retains a large JavaScript chunk warning. The production dependency audit reports four high advisories in the existing Firebase/Node gRPC dependency chain; this milestone does not force an unrelated Firebase downgrade or claim a clean dependency audit. No hosted acceptance, live upload, live rule inspection or live mutation was performed.

To create ONE intended real saree locally and verify that SAME product on Vercel:

1. Configure the approved server credentials/Cloudinary signed preset, publish reviewed rules/indexes and wait for readiness. Start both local commands in ordinary mode (not emulator mode). These next saves/uploads affect the real project; use the real product you intend to retain.
2. Sign in at `http://localhost:3000/admin`, Products & Inventory → Add New Saree. Enter the intended name, review its permanent URL slug, and Save product as Draft before completing it. It opens its saved edit URL; refresh and confirm persistence. Signed-out access to its public URL must show Product not found.
3. Select an enabled real category. Add SKU, description, fabric, weave/type, saree length, care and correct selling/original prices. Enter simple stock OR colour variants, never both. Include correct blouse details and optional weight. Select real product images, wait for verification, set descriptive alt text, ordering and cover. Save Draft and refresh to verify edits/images.
4. Choose Published and Save. Open `/product/{saved-slug}`, `/shop`, the corresponding `/collections/{category-slug}`, homepage and SKU search in a signed-out window. Verify INR, imagery/proportions, saree facts, selected variant stock and refresh. Online checkout, saved shopping and customer login remain unavailable.
5. Rename without changing the slug, adjust stock, unpublish, archive and re-publish as needed; confirm each saved state survives refresh and private products cannot be fetched publicly. Temporarily disabling its category must hide it too; restore the owner's intended state. A second editor's stale save should fail and retain its draft. Permanent deletion stays blocked; archive is the alternative.
6. After the owner releases the build and server variables on Vercel, sign in at `https://parinayasilks.vercel.app/admin`, edit the SAME product, and repeat public URL/refresh and publication checks. Do not create a duplicate test product or re-upload just for hosted acceptance.

Remaining live dependencies are trusted server credential configuration, Cloudinary setup and live acceptance, reviewed rules/index publication, actual Vercel plan/settings confirmation and owner deployment. Permanent product/media deletion awaits reliable historical reference checks. Stop before customer login, orders or payments.